Claude Code

AI

KB MinerU + LangExtract

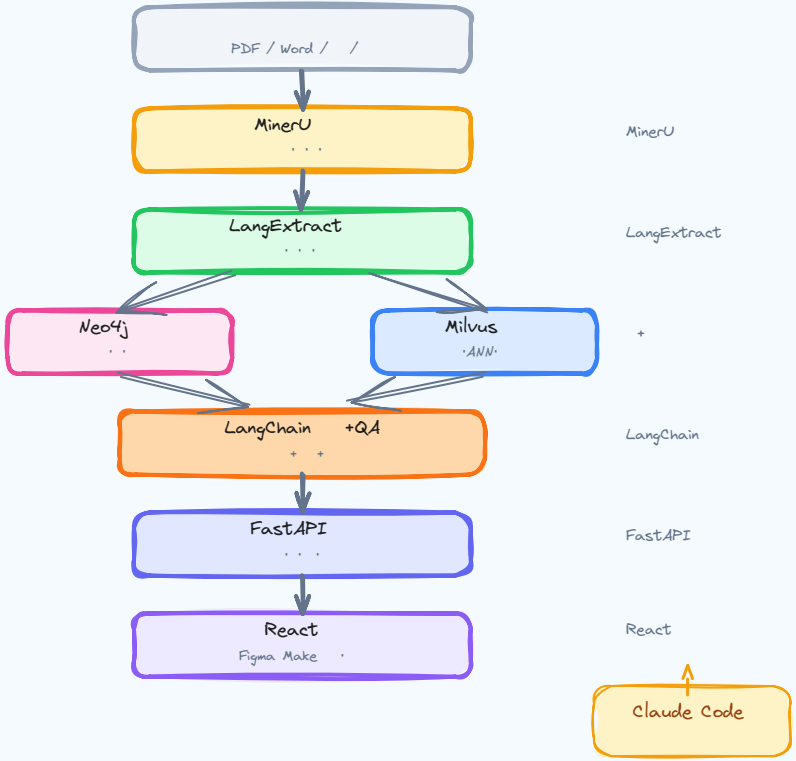
SP Spec
Claude Code

FM Figma Make
PRD → Figma → React

⚡ 18 Spec
MVP → → Pipeline → PRD → →

Claude Code + Cursor	Claude Opus 4.6	FastAPI + LangChain
React (Figma Make)	Neo4j + Milvus	MinerU + LangExtract

Spec



本期课程 学习收获

Claude Code × GraphRAG

Vibe Coding 能力进阶

5 项核心能力

01 Claude Code 实战操作能力

· 基础配置与权限模式
· Rules / Agent Skills / MCP 进阶定制
· Bash / Agent / 多模态交互
· 结合 GraphRAG 项目实战

核心能力：
从「会用工具」升级为「驾驭 AI 协作」

02 前后端设计规范能力

· 18阶段研发流水线全程实践
· 需求→设计→开发→验收闭环
· 接口契约 · 字段字典 · 状态机

实际价值：
让团队协作有章可循，不再口头对齐

03 GraphRAG 技术架构认知

· MinerU + LangExtract 数据管道
· Neo4j + Milvus 混合检索体系
· LangChain Agent 编排策略

核心认知：
AI 是理解复杂系统的加速器

04 Figma Make 驱动前端能力

· 需求快速可视化 · 交互原型生成
· 设计资产 → 开发资产映射
· Mock → 真实 API 无缝过渡

本质：从「画原型」升级为「生成可开发资产」

05 端到端工程化交付链路

· 从单点验证到系统联调到工程化发布
· Spec 驱动研发 · 可观测
· 可持续迭代的 AI 协作工程体系

最终目标
形成「指哪打哪」的
AI 工程化开发能力

课程总结

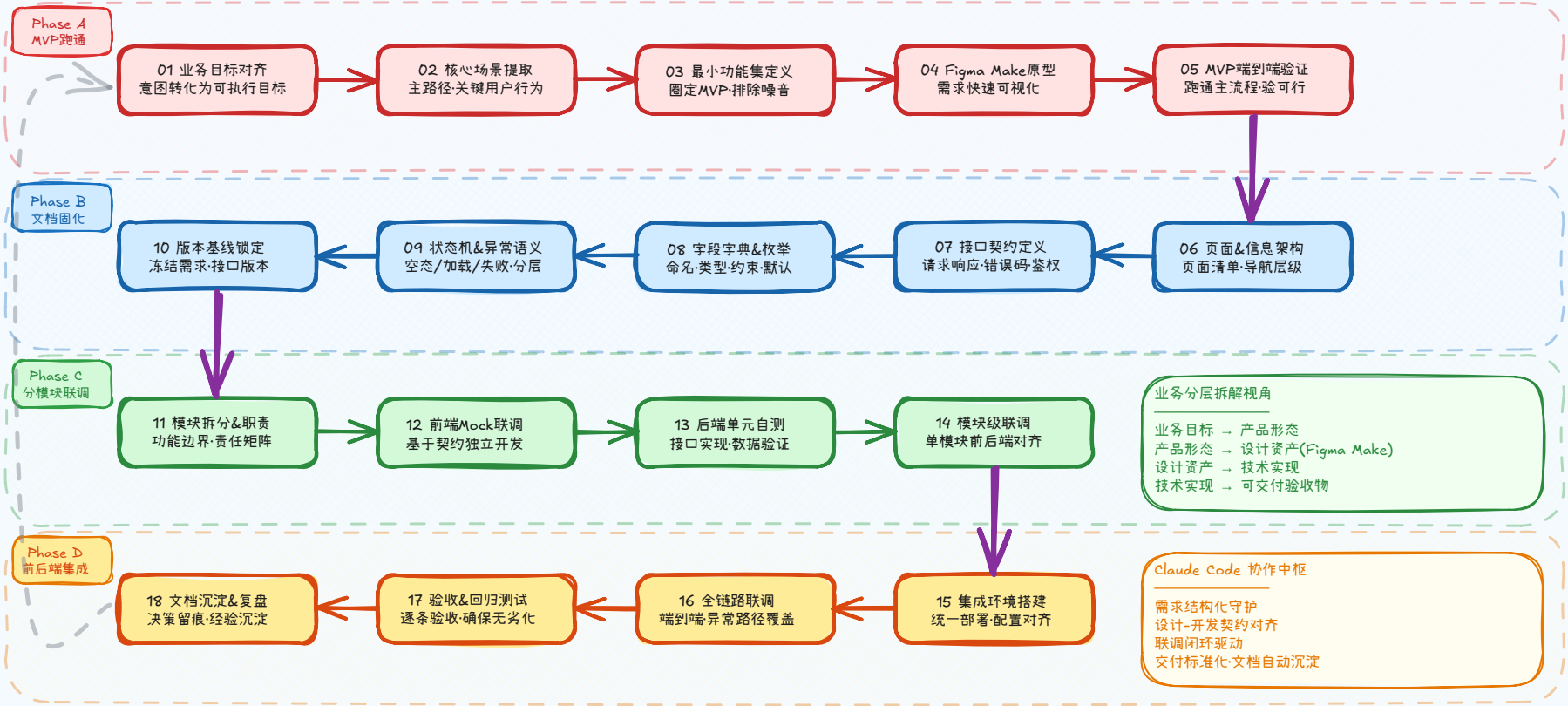
基础 → 规范 → 架构 → 设计 → 交付

本期课程不是教大家Claude Code工具的使用方法

而是教你如何驾驭 Claude Code + AI 完成真实项目工程化落地

Claude Code · 十八阶段研发流水线（前端驱动后端）

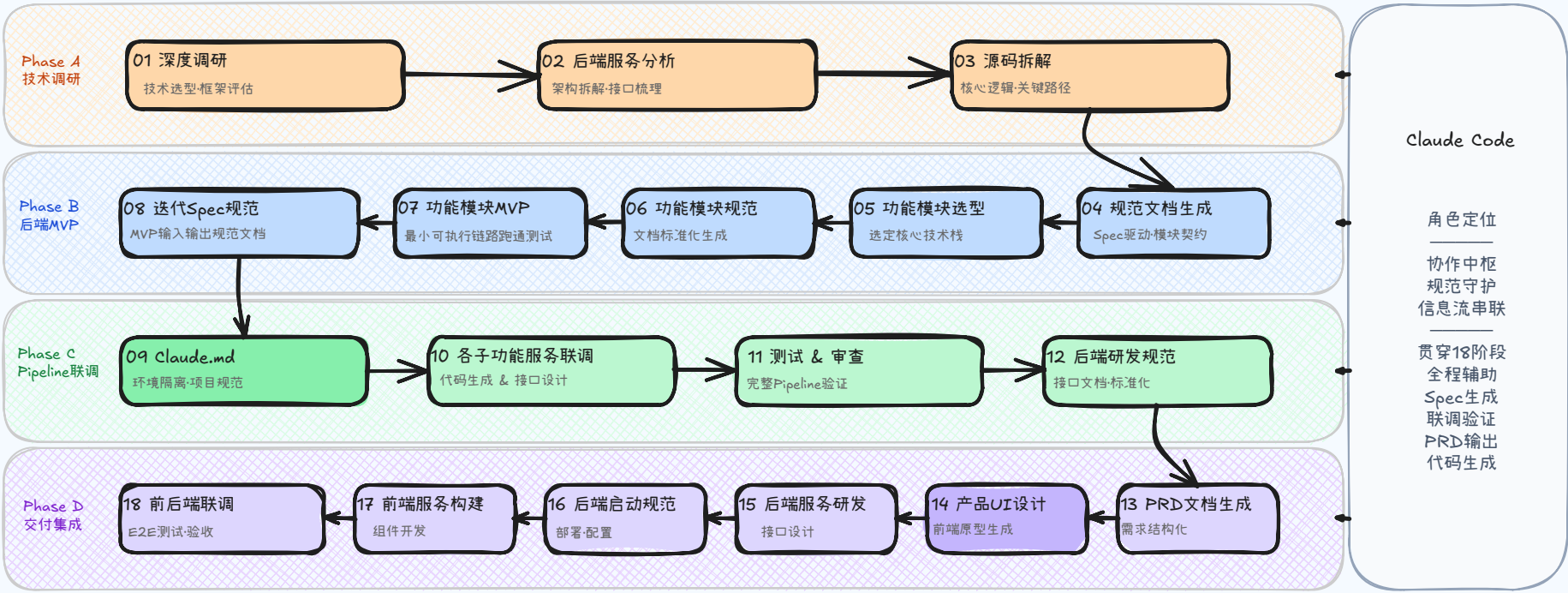
先MVP跑通 → 文档固化 → 分模块联调 → 前后端集成 · 从业务视角分层拆解到可运行产品



协作闭环：每轮复盘驱动下一次MVP迭代 | 需求 → 设计(Figma Make) → 开发 → 联调 → 验收 → 复盘

Claude Code · 前后端产品设计规范协作（后端驱动前端）

后端先行 → 规范固化 → 前端生成 → 前后端集成



协作闭环：技术调研 → 后端MVP逐个击破 → Pipeline联调 → PRD·前端生成 → 前后端集成 → 交付复盘

核心理念：后端先行 · Spec规范驱动 · Claude Code 全程协作 · MVP验证后推进

A 技术调研(1-3) → B 后端模块MVP(4-8) → C 后端Pipeline联调(9-12) → D PRD·前端·联调(13-18)

Claude Code · 多模态 GraphRAG 技术架构全景

端到端数据流 · 模块职责 · 技术栈映射 · 从单点验证到工程化交付

前端层

- **Figma Make**
  原型生成 · 交互基线 · 字段映射
- **React SPA**
  页面渲染 · 状态管理

Mock→真实API过渡 | 设计稿变更回流接口
交互预期驱动联调验收

服务层

- **FastAPI**
  路由 · 中间件 · OpenAPI Spec
- **Auth & 鉴权（可选）**
  Token · RBAC · 审计追踪

上传 · 解析 · 抽取 · 检索 · 问答 API
前后端契约对齐 · 接口版本管理

编排层

- **LangChain Agent**
  工具调用 · 推理链路 · ReAct策略
- **混合检索引擎**
  Keyword + Vector + Graph 融合排序
- **Celery + Redis（可选）**
  任务调度 · 异步队列 · 重试

存储层

- **Neo4j**
  知识图谱 · 路径推理
- **Milvus / pgvector**
  向量索引 · 语义召回
- **MinIO / S3 （可选）**
  对象存储 · 文件分层
- **Redis（可选）**
  缓存 · 会话 · 状态

数据管道

- **多源数据入口**
  PDF · HTML · Doc · Image
- **MinerU**
  文档解析 · 版面识别 · OCR
- **LangExtract**
  实体 · 关系 · 事件 · 属性抽取

→ 写入
存储层

横切关注点

- 可观测性
- 安全鉴权
- 评测体系
- 灰度发布
- 故障恢复
- 成本控制

Claude Code · 技术中枢角色

端到端 GraphRAG 流程编排入口
模块职责分配与依赖协调
Figma Make ↔ FastAPI 契约守护
数据流贯通：入口→解析→存储→检索
质量门禁：准确率/召回率/延迟
工程规范：CI/CD · 灰度 · 监控
异常治理：回滚 · 断点续跑 · 幂等
成本预算：模型/存储/计算资源

落地演进路线 · 5 Phases

Phase 1 · 单点验证
MinerU + LangExtract 管道跑通

Phase 2 · 检索闭环
Neo4j + Milvus 混合检索验证

Phase 3 · Agent 服务化
FastAPI + LangChain 编排上线

Phase 4 · 前端集成
Figma Make → React 全链路联调

Phase 5 · 工程化交付
可观测 + 灰度 + 容灾 + 成本治理

数据流：多源文档 → MinerU解析 → LangExtract抽取 → Neo4j/Milvus/MinIO存储 → 混合检索 → LangChain Agent → FastAPI → React
控制流：Claude Code 贯穿编排 · Figma Make驱动前端 · Celery调度异步 · Redis缓存加速 · 可观测性全链路覆盖

Claude Code 代理式编程工具概述

代理编码工具 · 读取代码 · 编辑文件 · 运行命令 · 集成开发工具

读取代码库

- 理解整个项目结构与上下文
- 自动分析文件间依赖关系
- 支持多语言代码索引与搜索
- 快速定位关键代码段
- 深入理解函数调用链
- 智能识别代码模式与规范

编辑文件

- 智能修改与重构代码
- 精确定位并修改目标代码
- 自动处理跨文件联动变更
- 保持代码风格一致性
- 批量重命名变量与函数
- 生成新文件与模块脚手架

运行命令

- 执行终端命令与自动化测试
- 运行构建、lint 与部署脚本
- 实时反馈命令执行结果
- 自动修复失败的测试用例
- 管理 Git 工作流与提交
- 安装依赖与环境配置

▼ 与开发工具深度集成 · 可在以下平台中使用 ▼

Terminal
原生命令行
最纯粹的交互

VS Code
官方扩展插件
内嵌编辑器体验

Cursor
AI 编辑器集成
无缝切换使用

Desktop App
独立桌面应用
沉浸式编码

Web
浏览器直接使用
无需安装

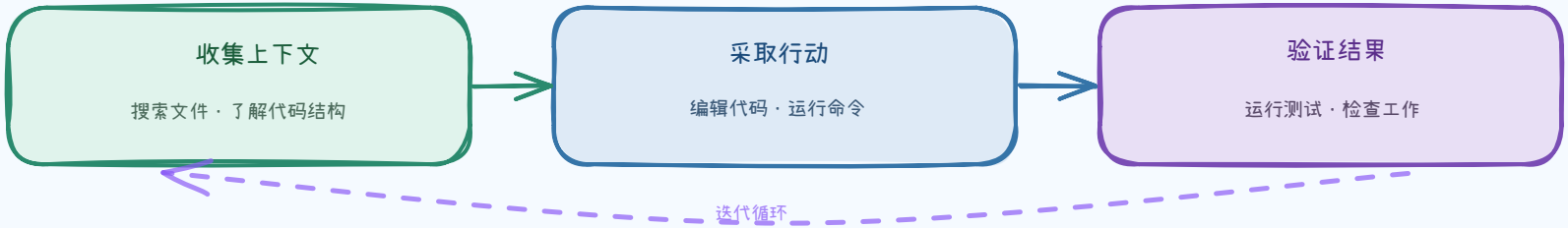
JetBrains
IDEA/WebStorm
全系列 IDE 支持

NEW
手机
随时随地编码
移动端新体验

Claude Code 是 Anthropic 推出的代理编码工具，直接在你的开发环境中运行
通过自然语言交互，帮助你理解代码、实现功能、修复 Bug、编写测试

Claude Code 如何工作

终端中的代理助手



重要使用说明

模型选择

`/model`

会话中随时切换模型

Sonnet
大多数编码任务首选

Opus
复杂架构决策的深度推理

claude --model <name>
启动时指定模型

扩展思考

`Ctrl+O`

切换详细模式查看推理过程

默认启用
复杂问题自动逐步推理

Opus 4.6 自适应推理
根据努力级别动态分配思考

适用场景
架构决策 · 调试 · 规划 · 权衡

权限模式

`Shift+Tab`

循环切换三种权限模式

默认模式
编辑和命令前询问确认

自动接受编辑
自动编辑文件，命令仍询问

Plan Mode
仅使用只读工具
创建可批准的执行计划

Claude Code 对话模式小技巧

01 · 中断和引导

- ⚡ 随时中断 Claude 的操作
- ✎ 输入更正内容，按 Enter
- 🔄 Claude 立即停止并调整
- ☑ 无需等待完成或重新开始

02 · 支持使用图像

添加图像的方式：

1. 拖放图像到 Claude Code 窗口
2. 复制图像 + Ctrl+V
3. 提供图像路径

🎯 适用：截图、UI 设计、图表
👆 Cmd / Ctrl+Click 打开引用图像

⚠ Windows 注意

使用 Alt+V
粘贴图像

03 · 使用 @ 引用文件和目录

使用 @ 快速包含文件或目录，直接注入上下文，无需等待读取

引用单个文件

```
> Explain the logic in @src/utils/auth.js
```

引用目录

```
> What's the structure of @src/components?
```

💡 善用这些技巧，让 Claude Code 成为你的高效编程伙伴

Claude Code 可观测闭环

看全局 → 做分析 → 盯执行

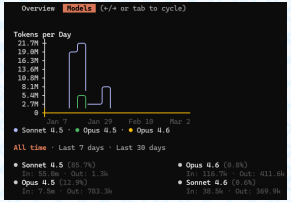
/stats

官方概览 · 内置面板

快速体检

核心能力
- 官方内置，零配置即用
- 会话活跃度 & Token 趋势
- 模型偏好 & 连续活跃统计
- 发现异常：波峰、token 突增

推荐场景
- 日常健康巡检
- 周度数据复盘入口



概览 · 巡检入口 · 认知现状

ccusage

https://github.com/ryoppippi/ccusage

深度分析 · 成本审计

深挖成因

核心能力
- 按日/周/月/会话多维分析
- 按计费窗口聚合成本
- 聚焦 Token 消耗与成本审计
- JSON 输出，自动化接入

推荐场景
- 精细化成本复盘
- 脚本化报表与趋势追踪

日期	主要模型	花费
01-26	gemini-2.5-flash	$0.00
02-21	opus-4-6, sonnet-4-6, opus-4.5-thinking	$9.21
02-22	MiniMax, haiku-4-5, opus-4-6, sonnet-4-6	$7.02
02-23	haiku-4-5, opus-4-6, sonnet-4-6	$10.68
02-24	haiku-4-5, opus-4-6	$22.47
02-25	opus-4-6	$0.07
总计		$49.44

细分 · 复盘分析 · 解释问题

ccstatusline

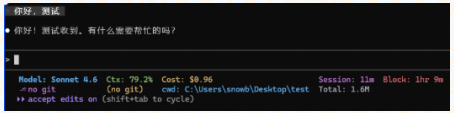
https://github.com/sirmalloc/ccstatusline

实时展示 · 状态可视化

实时反馈

核心能力
- 终端底部持续展示关键状态
- 模型/Token/成本/分支一览
- 高度可配置信息密度
- 可接入 ccusage 数据联动

推荐场景
- 编码过程实时感知
- 团队共享工作流监控



实时 · 过程监控 · 纠偏执行

协作落地路径

/stats 快速体检 → ccusage 深挖成因 → ccstatusline 指标前置 → 周期优化与告警

适用人群：个人开发者 | 团队负责人

建立「内置观察 + 离线分析 + 实时展示」的完整可观测体系

Claude Code 工具体系

没有工具只能文本回应 → 有了工具就能采取行动 — 每个工具调用形成反馈循环，驱动下一个决定

AGENT FEEDBACK LOOP

▼ 五大内置工具类别 ▼

文件操作

Read · Edit · Write

- 读取文件内容
- 精确编辑现有代码
- 创建全新文件
- 重命名和重新组织
- Notebook 编辑

搜索

Glob · Grep · Agent

- 按模式匹配查找文件
- 正则表达式搜索内容
- Agent 深度探索
- 快速定位目标代码

执行

Bash · Git · Process

- 运行 Shell 命令
- 启动服务器和测试
- Git 版本控制操作
- 管理后台任务进程

网络

WebSearch · WebFetch

- 搜索网络获取信息
- 获取在线文档内容
- 查找错误消息方案
- 访问最新 API 文档

代码智能

LSP · Diagnostics

- 实时类型错误警告
- 跳转到定义
- 查找所有引用
- 需要 LSP 插件支持

权限控制层

/allowed-tools

工具白名单机制

定义当前命令或技能执行时被允许
调用哪些工具（如 Bash、Read、Write）。

本质是白名单机制，让 Claude 在执行
命令时不能随意调用未列出的工具，
从而减少被滥用或误用的风险。

/permissions

权限规则管理

查看、管理、添加或删除权限规则：

- allow → 永久允许某些工具/命令
- deny → 永久拒绝某些工具/命令
- ask → 标记为需要用户确认

规则存储在 settings.json 或本地文件中。

Claude Code 工具权限模式关系图

四种权限模式

default

默认权限行为
未授权时询问用户

acceptEdits

自动批准文件更改
自动接受编辑请求

plan

规划/只读模式
仅分析规划不执行

⚠ DANGER

bypassPermissions

绕过权限检查(危险)
直接执行所有调用

设置方式

CLI 启动时指定
claude --permission-mode <mode>
为整个会话设定初始权限模式

会话内动态切换
Shift + Tab 循环切换模式
即时影响当前会话

权限评估链

规则 (Rules) —优先→ 权限模式 (Permission Mode) —决定→ 工具调用 (Tool Call)

⚡ 规则(Rules)优先于权限模式(Permission Mode)

模式定义了全局控制 Claude 工具使用方式的默认策略，在权限评估链中排在规则之后

Claude Code 工具权限关系

permission-mode（权限模式）

控制未明确允许/拒绝操作时的默认行为
并非规则本身，而是对默认决策行为的策略调节

启动时 / 内部切换

控制全局响应逻辑 ▼

/permissions（权限规则管理）

查看、管理、添加/删除真正的权限规则
规则决定 Claude 是否自动允许、拒绝或询问工具调用

✅ allow（永久允许） ❌ deny（永久拒绝） ❓ ask（询问）

settings.json / 本地规则文件 | 会话中随时可用

规则决定最终授权 ▼

allowed-tools / --allowedTools

在某一会话或一次性启动时
预先声明某些工具或工具类别被允许执行

启动 / 脚本调用 | 会话级规则

会话级覆盖

三者关系对比

permission-mode

定义规则：❌ 否
影响默认行为：✔ 是
使用时机：启动时 / 内部切换
特点：策略调节器，非规则本身

/permissions

定义规则：✔ 是（allow / deny / ask）
影响默认行为：✔ 是
使用时机：会话中随时可用
存储：settings.json / 本地文件
特点：真正的规则管理中心

allowed-tools / --allowedTools

定义规则：✔ 是（会话级）
影响默认行为：✔ 是
使用时机：启动 / 脚本调用
特点：临时性会话级预授权

决策流向

permission-mode → 设定默认策略
↓ /permissions → 定义具体规则
↓ allowed-tools → 会话级覆盖

permission-mode 调策略 → /permissions 定规则 → allowed-tools 临时授权

Claude Code Bash

Bash Shell

Option 1:

```
conda activate myenv
source venv/bin/activate

      claude

claude
```

Option 2: CLAUDE_ENV_FILE

```
export CLAUDE_ENV_FILE=
/path/to/env-setup.sh

  env-setup.sh

conda activate myenv
export MY_VAR=value
```

source

Option 3: SessionStart Hook

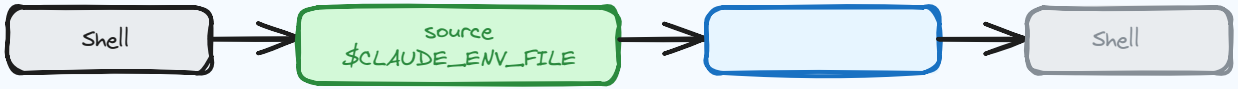
.claude/settings.json

```
hooks: { SessionStart: [{
command: "echo '...'
>> $CLAUDE_ENV_FILE"
}]}
```

Hooks Python

Bash

Shell → source $CLAUDE_ENV_FILE → → Shell



Claude Code Memory 核心概念总览

Memory 是什么

Claude Code 跨会话记忆有两类：

- 自动内存 (Auto Memory)
Claude 自动记录项目模式、调试经验、偏好

- CLAUDE.md 记忆
用户手写规则与指令

→ 两者都影响后续会话行为

怎么用（最短路径）

/init
初始化项目记忆
生成项目级 CLAUDE.md

/memory
查看并编辑内存文件
可开关自动内存

直接说"记住xxx" → 写入自动内存

记忆层级与优先级

- 组织级托管策略（系统路径）
- 项目级 ./CLAUDE.md
- 项目规则 ./.claude/rules/*.md
- 用户级 ~/.claude/CLAUDE.md
- 项目本地 ./CLAUDE.local.md（私有）

优先级递增

原则：更具体规则 → 优先级更高

关键机制

自动内存目录：
~/.claude/projects/<proj>/memory/

MEMORY.md 每次仅加载前 200 行
详细内容按需读取

Claude 递归查找：
从当前目录向上查找
CLAUDE.md / CLAUDE.local.md

子目录内 CLAUDE.md 为按需加载

200 行限制 | 递归加载 | 按需加载

@导入能力

支持 @path 导入
相对路径按"导入文件所在目录"解析

导入可递归，最大 5 跳

第一次遇到外部导入会弹一次批准

代码块/行内代码中的 @不会被当作导入

最佳实践

规则具体、可执行、可验证

保持模块化（.claude/rules）

定期清理和更新记忆
避免陈旧规则

可执行 & 可验证 | 模块化 rules

.claude/rules/*.md 项目规则详解

定位与价值

- 项目级、模块化规则机制
- 拆分大而散的 CLAUDE.md → 可维护的小规则文件
- 本质："可执行协作协议"，不是普通备注
- 适合团队协作：按主题沉淀规范
- 降低冲突和维护成本

生效机制

- .claude/rules/ 下所有 .md 文件自动加载
- 与 CLAUDE.md 同级（项目规则层优先级）
- 无 paths 条件 → 规则全局生效
- 有 paths 条件 → 仅匹配路径时生效
- 支持子目录递归发现

路径特定规则 (paths)　重点

- YAML frontmatter paths 字段
- glob: **/*.ts src/**/*
- 大括号: src/**/*.{ts,tsx}
- 多模式: {src,lib}/**/*.ts
- 无 paths = 全局规则
- 有 paths = 条件规则

```
---
paths:
- src/**/*.ts
- src/**/*.tsx
---
# 仅在匹配路径时
# 该规则才会生效
```

目录组织能力

- 子目录递归发现
rules/frontend/ rules/backend/
- 支持符号链接（可复用共享规则库）
- 内建循环链接检测，避免死循环
- 推荐按主题拆分：
code-style.md testing.md
security.md api-contract.md

优先级体系

~/.claude/rules/ 用户级（先加载）

覆盖 ▼

./.claude/rules/ 项目级（后加载，优先级更高）

原则："项目一致性优先"

进阶加载场景

- --add-dir 默认不加载额外目录规则
- 需设置环境变量：
CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1
- 适用：多仓协作 / 共享规范目录

编写与治理规范

- 单文件单主题
- 文件名语义化
- 规则要「具体、可执行、可验证」
- 条件规则谨慎使用
- 定期审查，清理过期约束

常见误区

所有规则塞进一个文件 → 难维护，冲突频繁
paths 写得过宽 → 造成误命中，规则在不该生效时生效
规则只讲原则，不讲可执行标准 → 无法落地
忽略优先级与覆盖关系 → 团队口径不一致，行为不可预测

Claude Code 实战经验分享

每一步都要求：可执行 · 可验证 · 有文档产出

核心原则：先跑通 MVP 再扩展
大道至简 · 遇到问题再解决

可执行 · 可验证 · 有文档

先 MVP 再扩展

原则一：一定要给验证方式

核心

提供验证标准，让 Claude 验证自己的工作

运行测试 · 比较截图 · 验证输出
Claude 能自我验证时，表现显著提高

方法

每个阶段都要有明确的验证产出

MVP 能跑 → 接口能调 → 前端能渲染 → E2E通过

实战例

模块开发 → 写测试用例 → 跑通测试
接口开发 → Postman/curl 验证响应
前端页面 → 截图对比设计稿
联调阶段 → E2E 自动化测试覆盖

不要写完就算，一定要跑通验证，验证是质量的唯一保障

原则二：不要盲目 Plan 后就生成

正确姿势

调研 + DeepResearch → 提炼 → 再 Plan

没思路时：先和 Claude 聊天，提炼关键点
有思路后：再给出结构化的 Plan 指令

上下文

梳理技术架构 + 各阶段输入输出规范

架构清晰 → 模块边界明确 → 利于后续扩展

流程

调研 → 聊天 → 提炼 → Plan → 执行

每一步都有产出，不是一步到位

宁可多花时间调研和聊天，也不要盲目让 AI 自动实现

原则三：提供充分上下文

前端规范

先产出详细 PRD → 再做 Figma 设计
在 Figma 中调试好 → 再导入后端对接

接口契约

前后端接口不一致时，联调前必须确定
到底以前端接口为准，还是以后端接口为准

架构先行

自己梳理好技术架构
做好各阶段的输入输出规范 → 利于后续扩展

实战教训

不要盲目追求高阶功能，大道至简！

先用最简单的方式 → 遇到问题再解决
例：模型总忘启动命令 → 才去配 CLAUDE.md

上下文越充分，AI 输出质量越高，规范是协作的基石

黄金法则：从简单开始，逐步演进

第一步：跑通最简 MVP
能跑 > 完美

→

第二步：发现真实问题
问题驱动 > 预设方案

→

第三步：按需引入工具
如 CLAUDE.md / Hooks / MCP

大道至简
遇到问题再解决！

Boris Cherny
@bcherny

I'm Boris and I created Claude Code. Lots of people have asked how I use Claude Code, so I wanted to show off my setup a bit.

My setup might be surprisingly vanilla! Claude Code works great out of the box, so I personally don't customize it much. There is no one correct way to use Claude Code: we intentionally build it in a way that you can use it, customize it, and hack it however you like. Each person on the Claude Code team uses it *very* differently.

So, here goes.

我是 Boris，我创建了 Claude Code。很多人问我如何使用 Claude Code，所以我想展示一下我的设置。

我的设置可能出乎意料地朴素！Claude Code 开箱即用效果很好，所以我个人很少自定义。使用 Claude Code 没有唯一正确的方式：我们有意把它构建成你可以随意使用、定制和改造的方式。Claude Code 团队的每个人使用方法都很不同。

那么，开始。